

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
ПЕНЗЕНСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ПОЛИТЕХНИЧЕСКИЙ ИНСТИТУТ
ФАКУЛЬТЕТ ВЫЧИСЛИТЕЛЬНОЙ ТЕХНИКИ

Утвержден на заседании кафедры
«Вычислительная техника»

"__" _____ 20__ г.

Заведующий кафедрой
_____ М.А. Митрохин

ОТЧЕТ ПО УЧЕБНОЙ (ЭКСПЛУАТАЦИОННОЙ) ПРАКТИКЕ

(2025/2026 учебный год)

Цыбузин Даниил Владиславович

Направление подготовки 09.03.01 «Информатика и вычислительная техника»

Форма обучения – очная Срок обучения в соответствии с ФГОС – 4 года

Год обучения _____ 2 _____ семестр _____ 4 _____

Период прохождения практики с 19.06.2026 по 16.07.2026

Кафедра «Вычислительная техника»

Заведующий кафедрой д.т.н., профессор, Митрохин М.А.
(должность, ученая степень, ученое звание, Ф.И.О.)

Руководитель практики к.т.н., доцент, Карамышева Н.С.
(должность, ученая степень, ученое звание)

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
ПЕНЗЕНСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ПОЛИТЕХНИЧЕСКИЙ ИНСТИТУТ
ФАКУЛЬТЕТ ВЫЧИСЛИТЕЛЬНОЙ ТЕХНИКИ

Утвержден на заседании кафедры
«Вычислительная техника»
"___" _____ 20__ г.
Заведующий кафедрой
_____ М.А. Митрохин

**ИНДИВИДУАЛЬНЫЙ ПЛАН ПРОХОЖДЕНИЯ УЧЕБНОЙ
(ЭКСПЛУАТАЦИОННОЙ) ПРАКТИКИ**

(2025/2026 учебный год)

Цыбузин Даниил Владиславович

Направление подготовки 09.03.01 «Информатика и вычислительная техника»

Форма обучения – очная Срок обучения в соответствии с ФГОС – 4 года

Год обучения 2 семестр 4

Период прохождения практики с 19.06.2026 по 16.07.2026

Кафедра «Вычислительная техника»

Заведующий кафедрой д.т.н., профессор, Митрохин М.А.
(должность, ученая степень, ученое звание, Ф.И.О.)

Руководитель практики к.т.н., доцент, Карамышева Н.С.
(должность, ученая степень, ученое звание)

№ п/п	Планируемая форма работы во время практики	Количество часов	Календарные сроки проведения работы	Подпись руководителя практики от вуза
1	Выбор темы и разработка индивидуального плана проведения работ	26	19.06.26 – 24.06.26	
2	Подбор и изучение материала по теме работы	26	24.06.26 – 26.06.26	
3	Установка виртуальной машины	26	26.06.26 – 28.06.26	
4	Установка операционной системы	30	01.07.26 – 04.07.26	
5	Разработка программы на языке Си	30	04.07.26 – 09.07.26	
6	Тестирование и отладка	38	09.07.26 – 12.07.26	
7	Оформление отчёта	40	12.07.26 – 16.07.26	
	Общий объём часов	216		

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
ПЕНЗЕНСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ПОЛИТЕХНИЧЕСКИЙ ИНСТИТУТ
ФАКУЛЬТЕТ ВЫЧИСЛИТЕЛЬНОЙ ТЕХНИКИ

ОТЧЁТ

О ПРОХОЖДЕНИИ УЧЕБНОЙ (ЭКСПЛУАТАЦИОННОЙ) ПРАКТИКИ

(2025/2026 учебный год)

Цыбузин Д.В.

Направление подготовки 09.03.01 «Информатика и вычислительная техника»

Форма обучения – очная Срок обучения в соответствии с ФГОС – 4 года

Год обучения 2 семестр 4

Период прохождения практики с 19.06.2026 по 16.07.2026

Кафедра «Вычислительная техника»

1. Цыбузин Д.В. выполнял практическое задание «Двоичная сортировка (binary tree sort)». На первоначальном этапе была установлена виртуальная машина, затем ОС Linux (Ubuntu).

Бакалавр Цыбузин Д.В. _____ " ____ " _____ 2026 г.

Руководитель Карамышева Н.С. _____ " ____ " _____ 2026 г.
практики

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
ПЕНЗЕНСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ПОЛИТЕХНИЧЕСКИЙ ИНСТИТУТ
ФАКУЛЬТЕТ ВЫЧИСЛИТЕЛЬНОЙ ТЕХНИКИ

ОТЗЫВ

О ПРОХОЖДЕНИИ УЧЕБНОЙ (ЭКСПЛУАТАЦИОННОЙ) ПРАКТИКИ

(2025/2026 учебный год)

Цыбузин Даниил Владиславович

Направление подготовки 09.03.01 «Информатика и вычислительная техника»

Форма обучения – очная Срок обучения в соответствии с ФГОС – 4 года

Год обучения 2 семестр 4

Период прохождения практики с 19.06.2026 по 16.07.2026

Кафедра «Вычислительная техника»

В процессе выполнения практики Цыбузин Д.В. решал следующие задачи:

- 1) Разработка двоичной сортировки;
- 2) Установка виртуальной машины, ОС Linux и работа с ними;
- 3) Подготовка отчета по выполненной практической работе.

За период выполнения практики были освоены:

- 1) Работа с виртуальной машиной;
- 2) Язык программирования Python;
- 3) Изучен способ установки ОС на виртуальную машину и ее первоначальная настройка.

4) Во время выполнения работы Цыбузин Д.В. показал себя ответственным, добросовестным учеником, знающим свой предмет, имеющим представление о современном состоянии науки, владеющим современными общенаучными знаниями по информатике и вычислительной технике.

За выполнение работы Цыбузин Д.В. заслуживает оценки «_____».

Руководитель практики к.т.н., доцент, Карамышева Н.С. « _____ » 2026 г.

Содержание

Введение.....	10
1. Установка ОС Linux на виртуальную машину	12
1.1. Установка виртуальной машины VirtualBox	12
1.2. Создание виртуальной машины	16
1.3. Установка ОС Ubuntu на виртуальную машину.....	18
1.4. ОС Ubuntu	24
2. Интегрированная среда разработки VS Code.....	25
2.1. Описание VS Code	25
2.2. Описание установки	27
3. Разработка программы на языке Python в среде разработки VS Code.....	29
3.1. Формулировка задачи.....	29
3.2. Работа с GitHub	29
3.3. Создание проекта в VS Code	29
3.4. Описание программы	32
Заключение.....	36
Список использованных источников и литературы	37
Приложение А Листинг программы.....	38
Приложение Б Результаты испытаний.....	44

Введение

Linux, или GNU/Linux, представляет собой семейство Unix-подобных операционных систем, построенных на основе ядра Linux и включающих набор утилит и программ проекта GNU. Данная операционная система имеет множество дистрибутивов, среди которых наиболее популярными являются Ubuntu, Linux Mint, Fedora, Debian и другие. Большинство из них распространяются совершенно бесплатно с открытым исходным кодом, что делает их особенно привлекательными для разработчиков программного обеспечения. Среди ключевых преимуществ ОС Linux можно выделить высокий уровень безопасности, устойчивость к вирусным угрозам и вредоносному программному обеспечению. Популярные дистрибутивы Linux отличаются простотой установки и нетребовательностью к аппаратным ресурсам компьютера. Данная операционная система идеально подходит для программистов и веб-разработчиков благодаря наличию встроенных компиляторов, интерпретаторов и удобной командной строки, через которую можно устанавливать практически любые приложения. Linux успешно функционирует как на домашних компьютерах, так и на рабочих станциях и серверах.

Виртуальная машина (ВМ или VM) — это виртуальный компьютер, который использует выделенные ресурсы реального компьютера (процессор, диск, адаптер). Эти ресурсы хранятся в облаке и позволяют ВМ работать автономно. ВМ может работать в отдельном окне как программа или запускаться через панель управления. Виртуальная машина может использоваться для развертывания двух и более операционных систем на одном физическом устройстве. Таким образом, установив на виртуальную машину операционную систему Linux, можно работать в ней, не перезагружая компьютер с основной ОС Windows. К тому же ВМ дает возможность одновременной работы в нескольких операционных системах, а также взаимодействие между ними. Достоинством виртуальной машины является то, что работа в ней никак не навредит реальному компьютеру.

Задачей данной практической работы стала установка операционной системы Linux на виртуальную машину и разработка программы на языке Python в установленной в данной ОС интегрированной среде разработки.

1. Установка ОС Linux на виртуальную машину

1.1. Установка виртуальной машины VirtualBox

Virtualbox — кроссплатформенное программное обеспечение, обладающее понятным и дружелюбным интерфейсом. Для установки виртуальной машины нужно скачать с официального сайта установочный файл и запустить его.

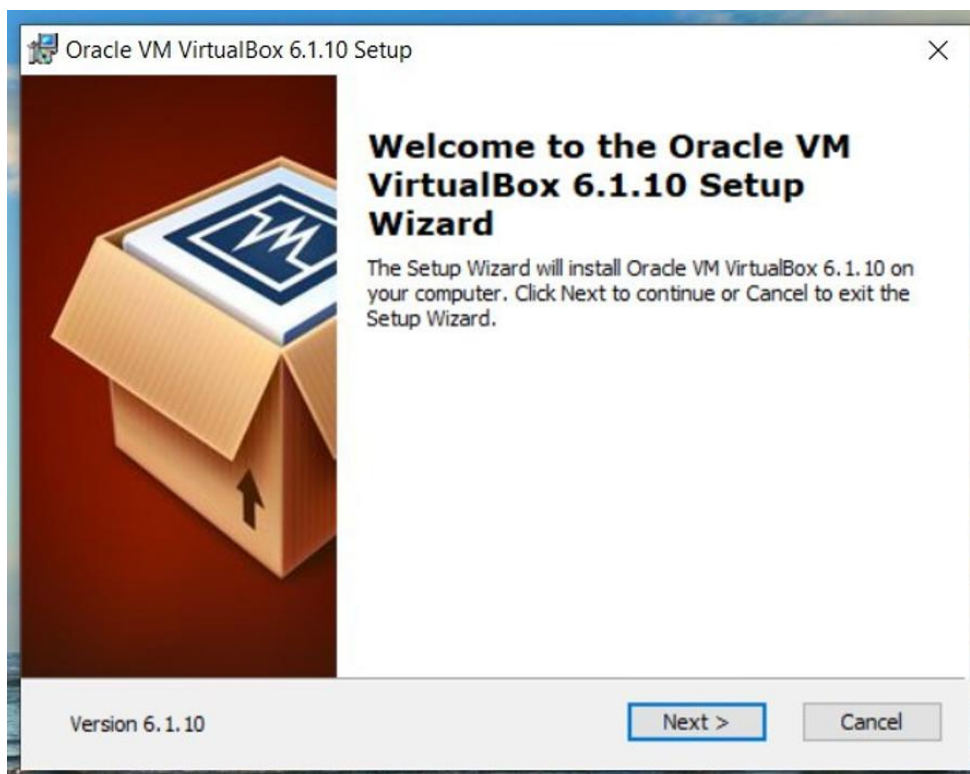


Рисунок 1 — Запуск установочного файла VirtualBox
Затем выбираем компоненты для установки и расположение.

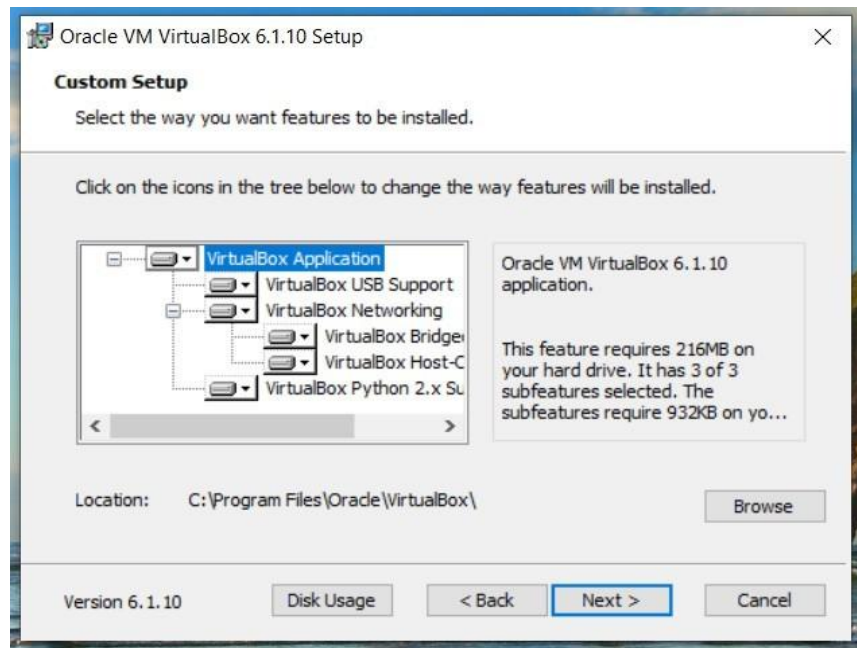


Рисунок 2 — Выбор компонентов для установки и расположения
Далее отмечаем «Создать ярлык на рабочем столе».

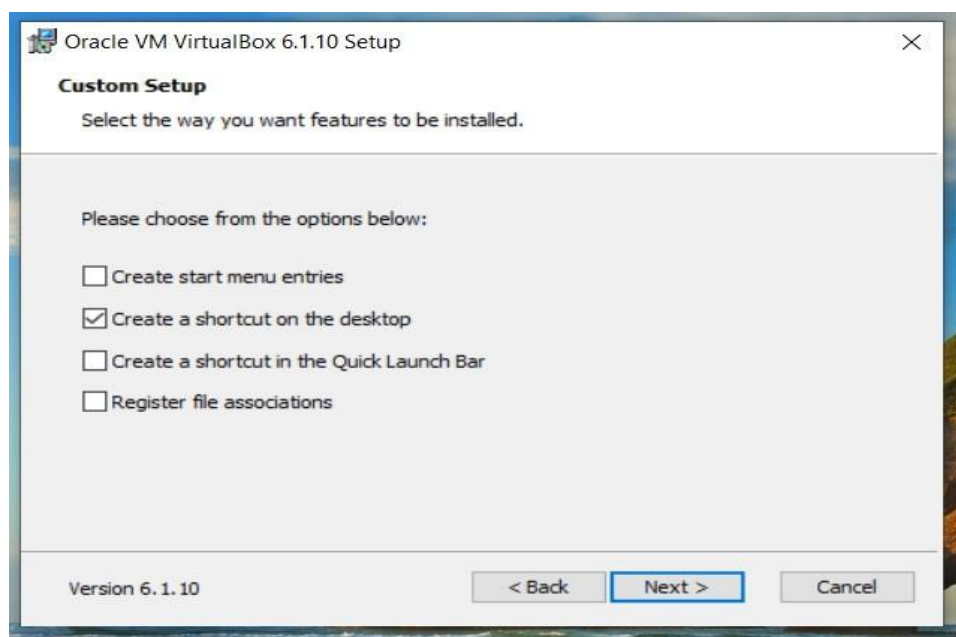


Рисунок 3 — Создание ярлыка на рабочем столе

Потом соглашаемся на отключение Интернет-соединения на время установки VirtualBox.



Рисунок 4 — Временное отключение Интернет-соединения

Затем устанавливаем программное обеспечение для виртуальной машины.

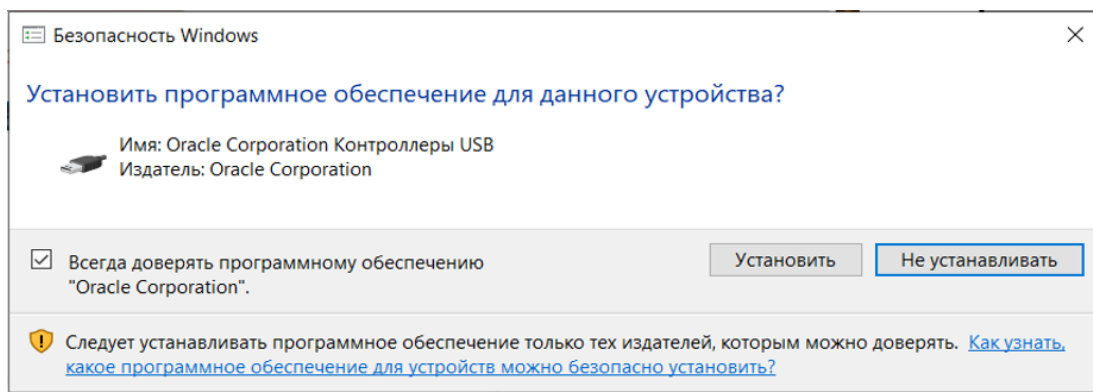


Рисунок 5 — Установка программного обеспечения для ВМ

Далее нажимаем кнопку «Завершить» для выхода из Мастера установки.



Рисунок 6 — Выход из Мастера установки

Затем устанавливаем VirtualBox.

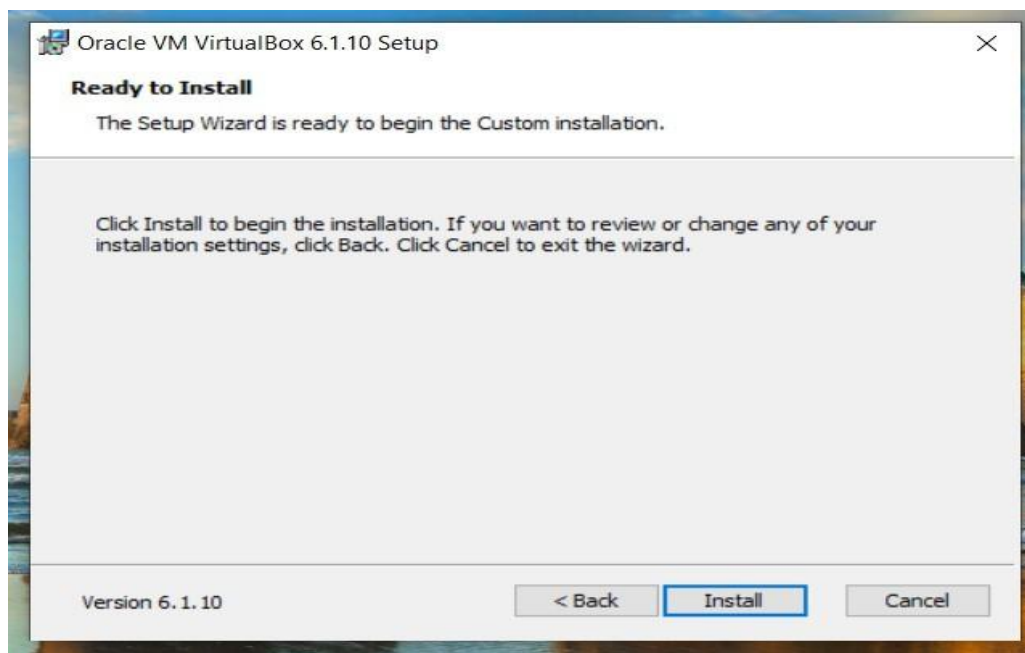


Рисунок 7 — Установка VirtualBox

Установка VirtualBox успешно завершена, для проверки запускаем ВМ.

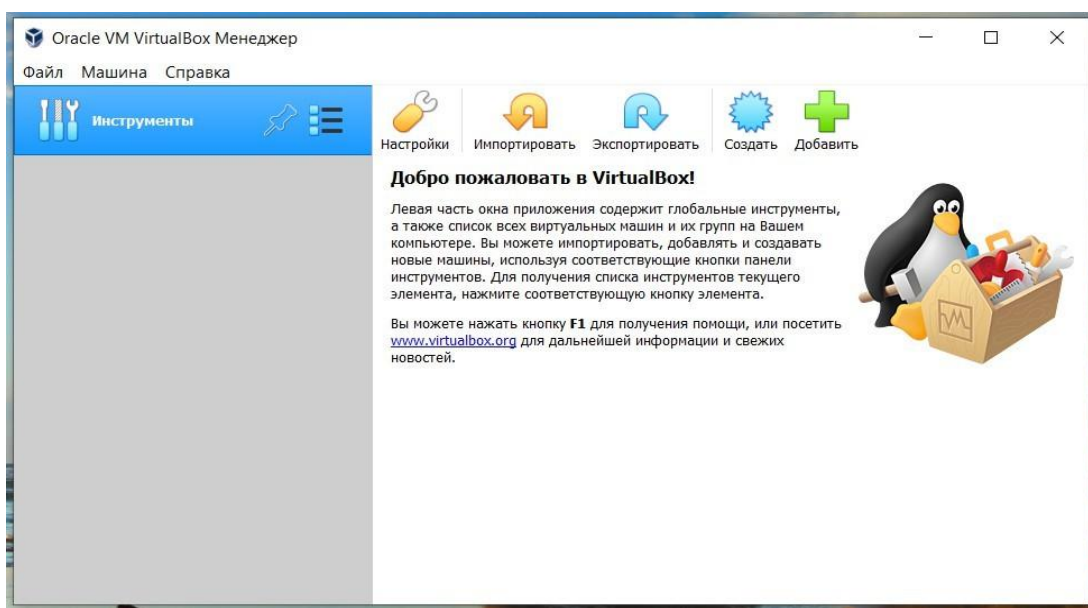


Рисунок 8 — Запуск VirtualBox

1.2. Создание виртуальной машины

Для создания виртуальной машины запускаем VirtualBox (рисунок 8) и нажимаем кнопку «Создать».

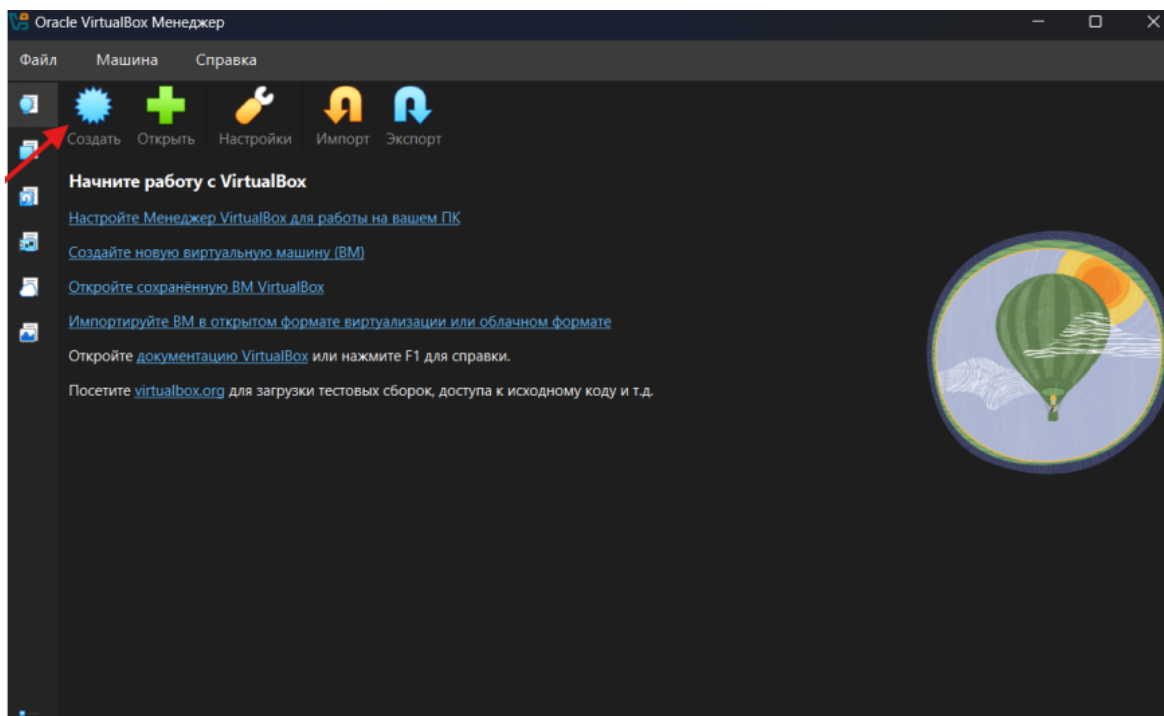


Рисунок 9 — Создание виртуальной машины

Затем указываем имя, местоположение, тип и версию операционной системы новой виртуальной машины.

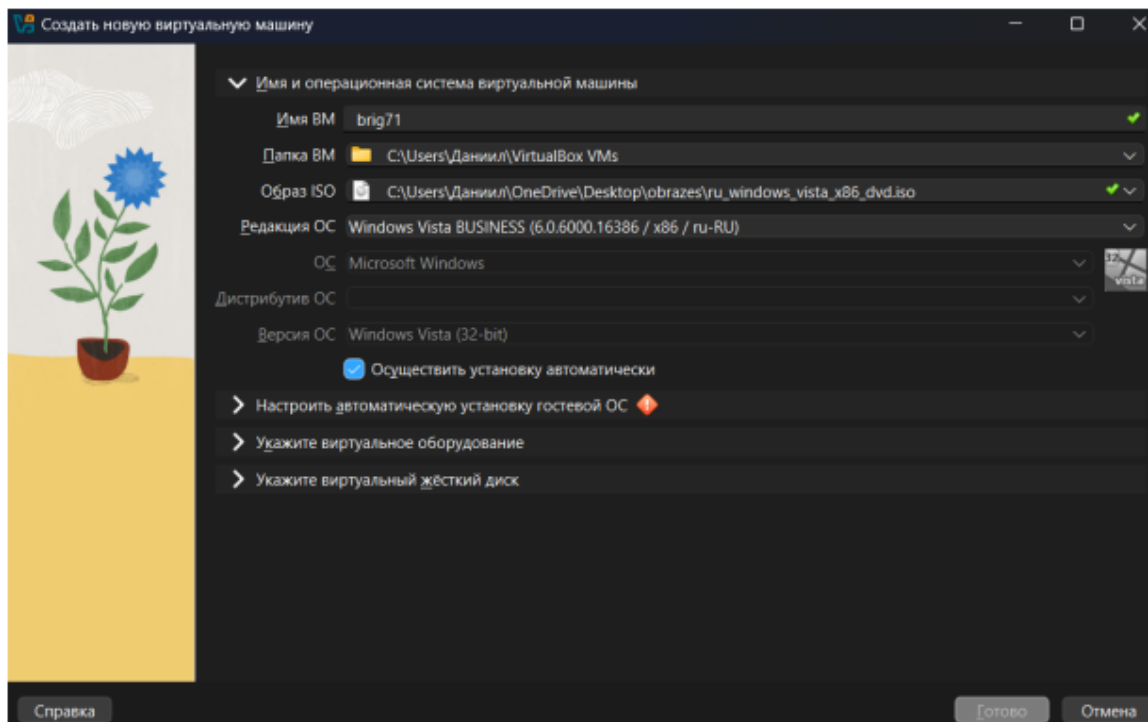


Рисунок 10 — Указание имени и типа ОС

Далее указываем объем оперативной памяти виртуальной машины.

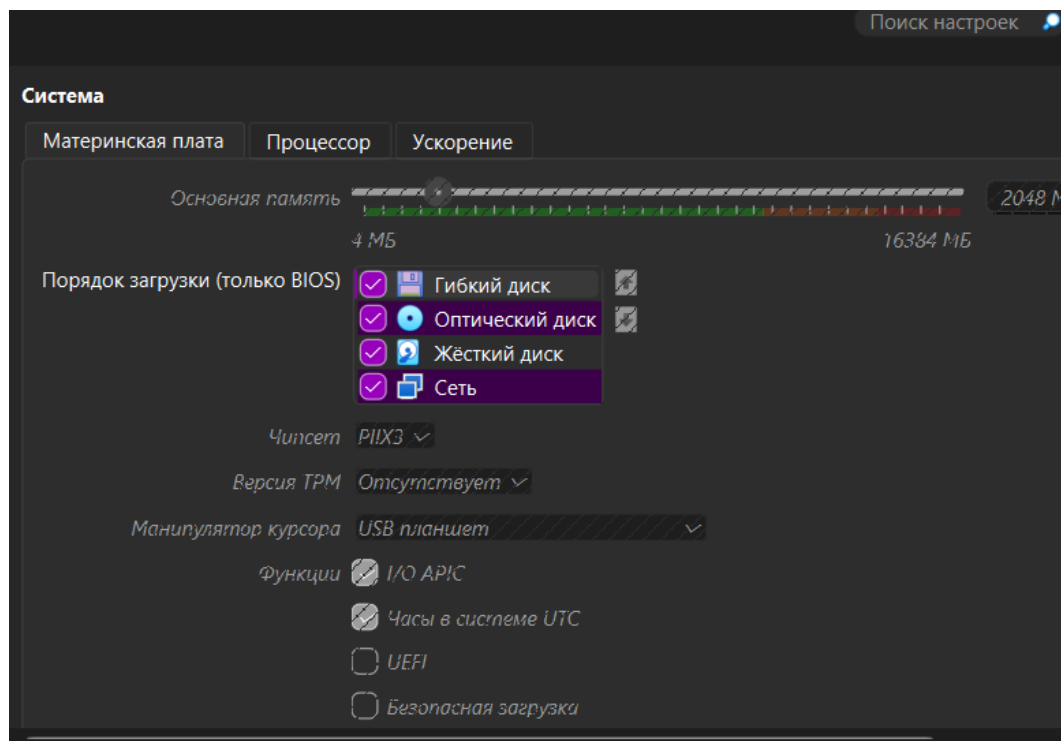


Рисунок 11 — Объем оперативной памяти

Потом создаем новый виртуальный жесткий диск.

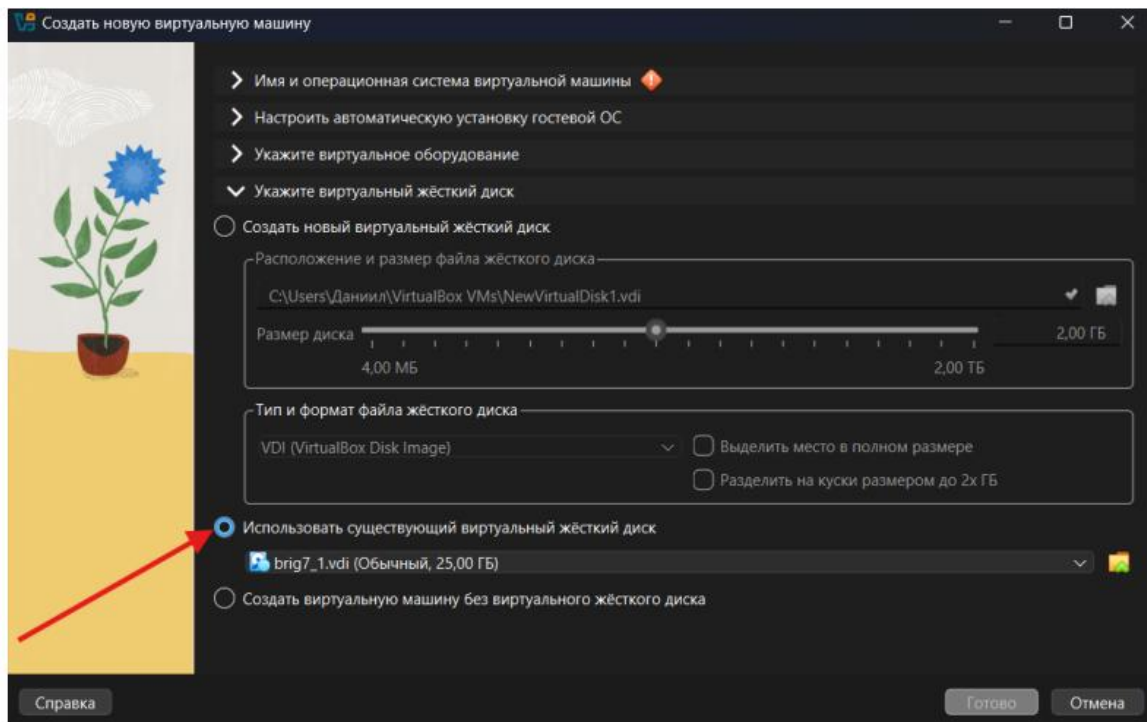


Рисунок 12 — Создание жесткого диска

Виртуальная машина «Ubuntu» создана.

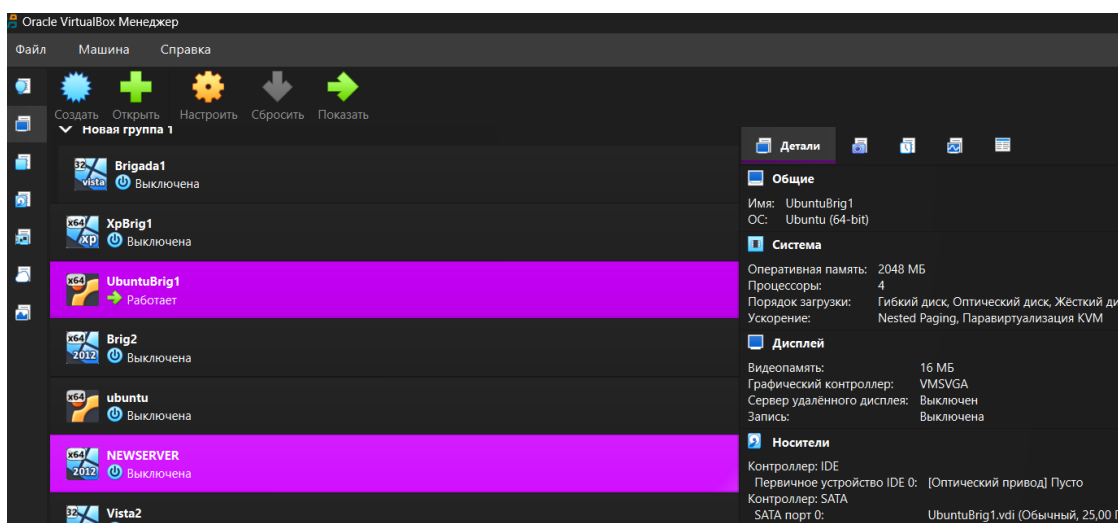


Рисунок 13 — Виртуальная машина «Ubuntu»

1.3. Установка ОС Ubuntu на виртуальную машину

Для установки ОС Ubuntu нужно скачать с официального сайта файл, содержащий дистрибутив операционной системы, и запустить созданную виртуальную машину «Ubuntu».

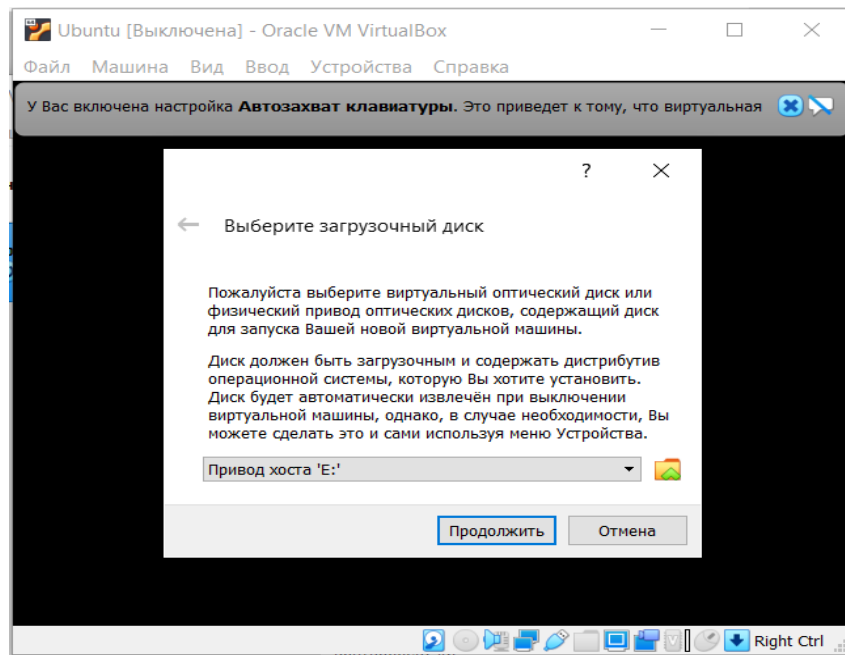


Рисунок 14 — Запуск виртуальной машины «Ubuntu»

Затем выбираем скачанный файл образа диска с ОС Ubuntu.

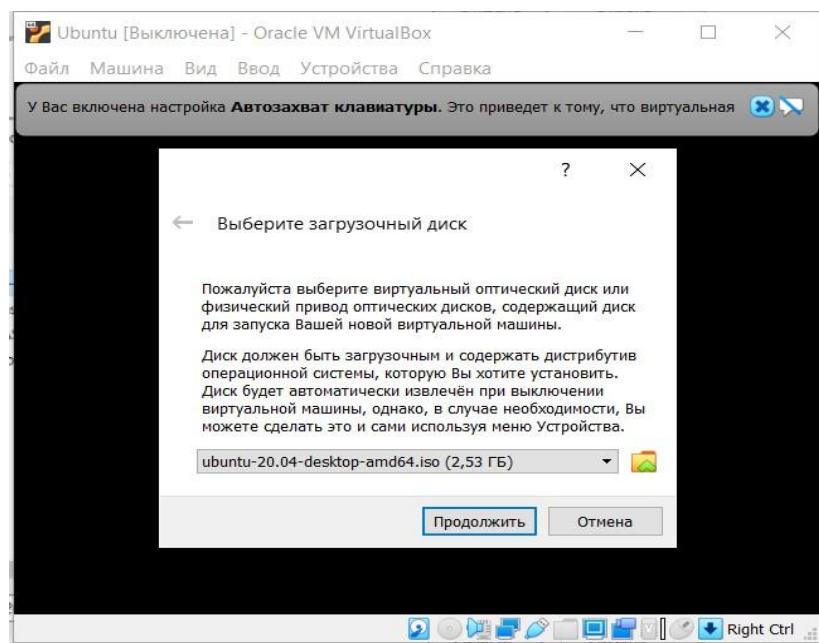


Рисунок 15 — Выбор загрузочного диска

Далее запускается установщик Ubuntu, и выбираем «Установить Ubuntu».

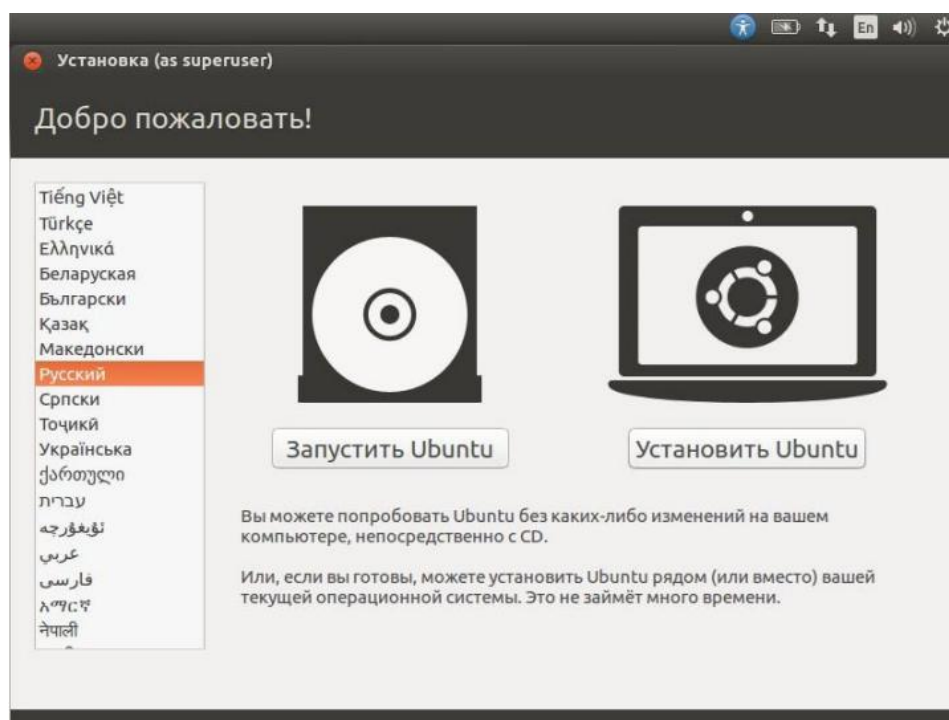


Рисунок 16 - Запуск установщика Ubuntu

Потом выбираем обычную установку приложений и их обновлений.

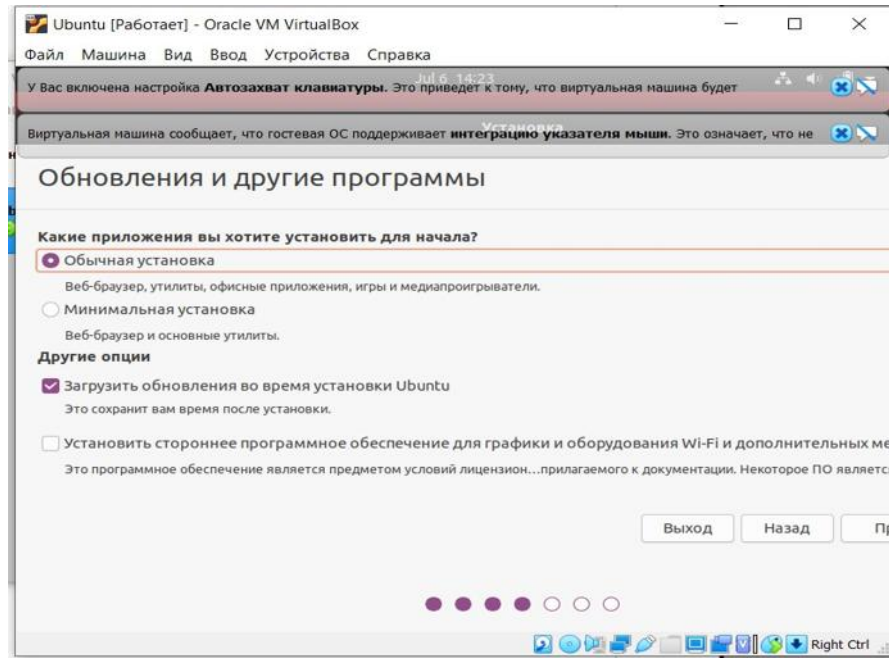


Рисунок 17 — Установка приложений и их обновлений

Затем указываем тип установки Ubuntu.

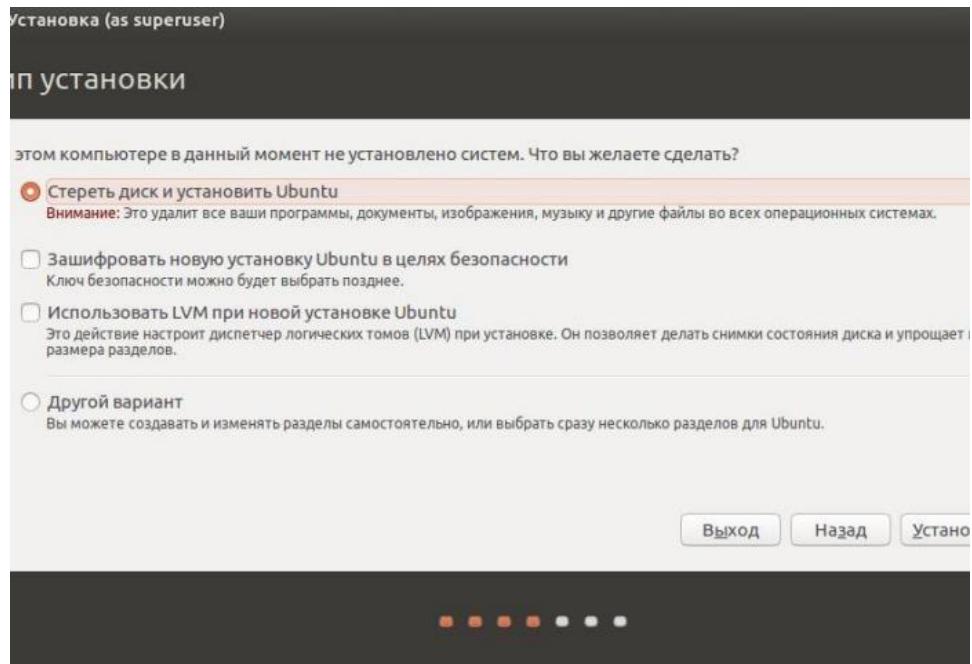


Рисунок 18 — Тип установки Ubuntu

Далее задаем имя компьютера, пользователя и указываем пароль для входа в систему.

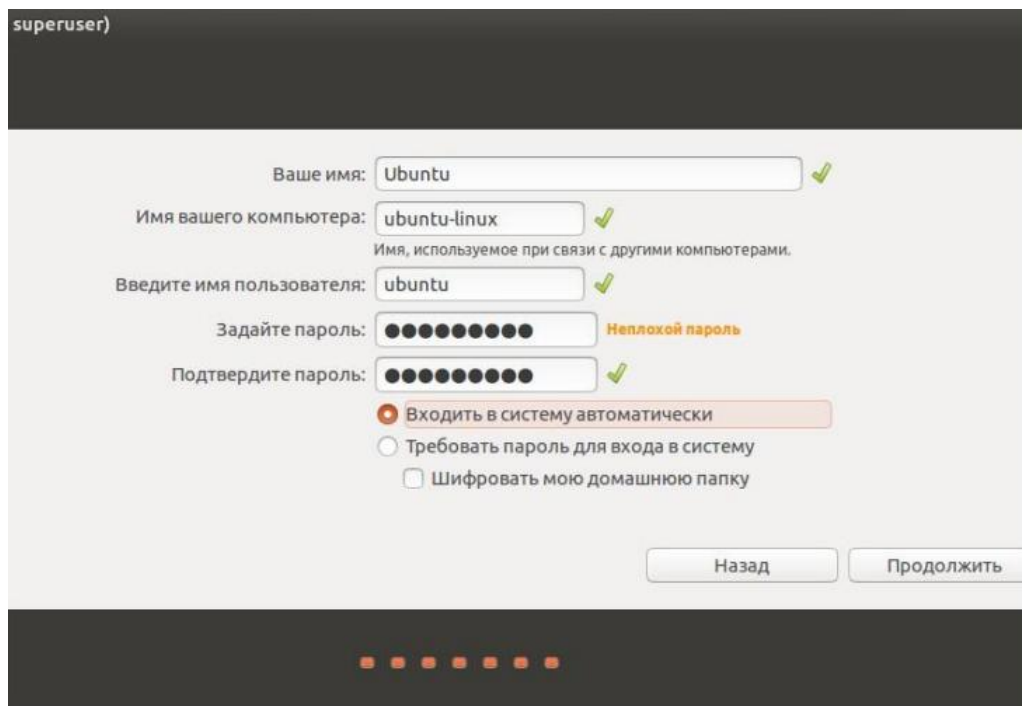


Рисунок 19 — Указание имени и пароля пользователя

Потом осуществляется установка Ubuntu.

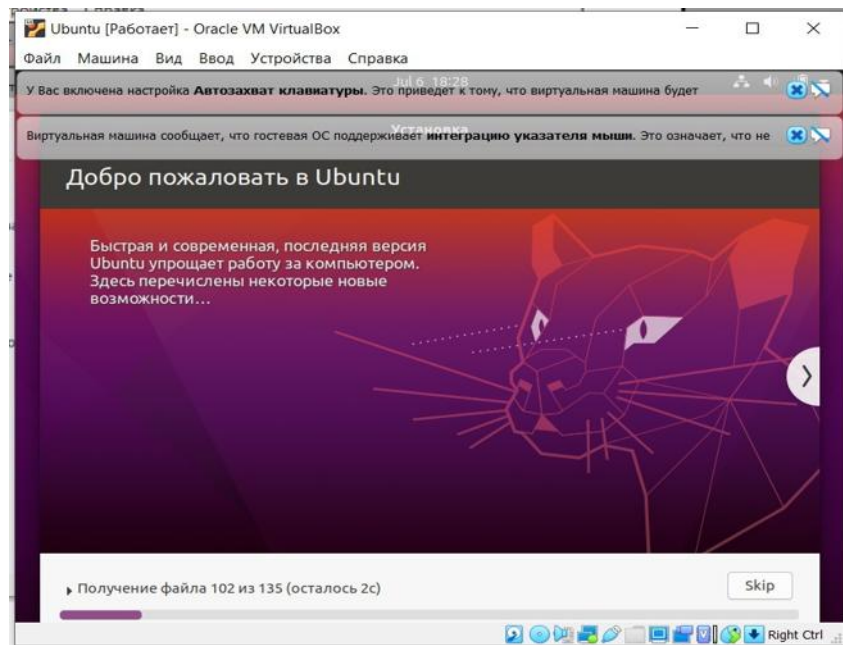


Рисунок 20 — Установка Ubuntu

Установка операционной системы Ubuntu завершена, выбираем перезагрузку виртуальной машины.

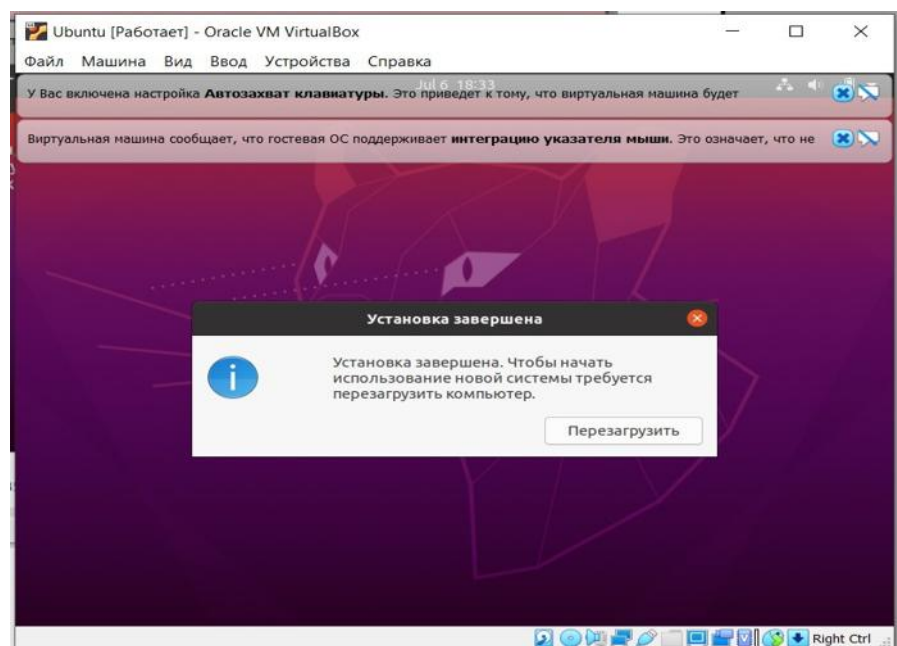


Рисунок 21 — Перезагрузка виртуальной машины

Затем запускается Ubuntu, вводим пароль для входа в систему.

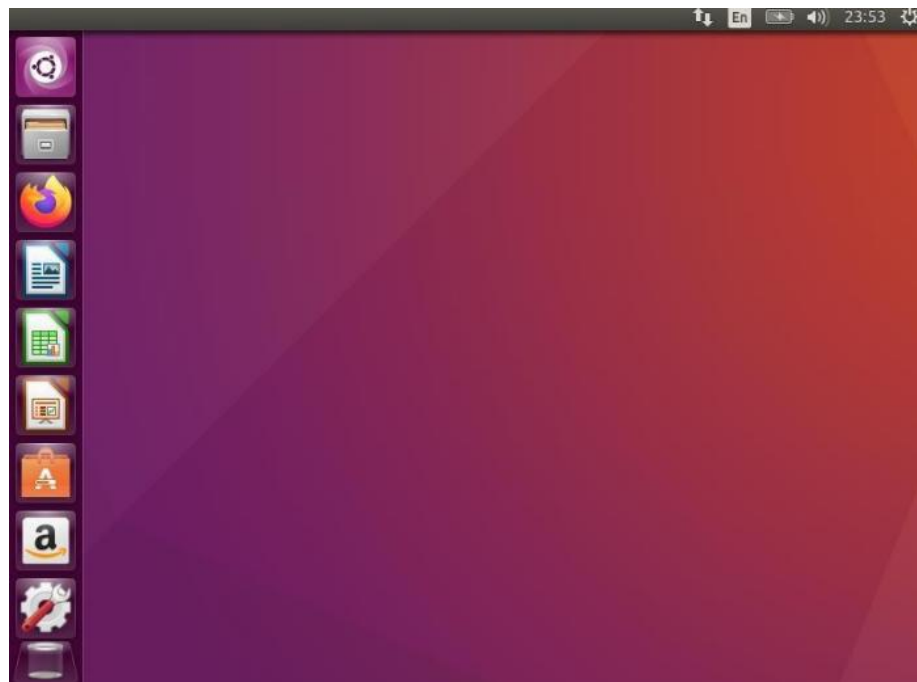


Рисунок 22 — ОС Ubuntu (ее рабочий стол)

1.4. ОС Ubuntu

Ubuntu — операционная система, основанная на Debian GNU/Linux, является одним из самых популярных дистрибутивов Linux. Она ориентирована на удобство и простоту использования, включает широко распространённое использование утилиты `sudo`, которая позволяет пользователям выполнять администраторские задачи. Ubuntu, кроме того, имеет развитую интернационализацию./4/ К главным особенностям дистрибутива Ubuntu можно отнести:

- 1) стабильность работы — не требует частых перезагрузок компьютера;
- 2) безопасная система, не требующая антивирусов;
- 3) бесплатная операционная система;
- 4) приятный и понятный интерфейс, большое количество бесплатных тем и эффектов;
- 5) быстрая и простая установка, даже для неопытных пользователей, основное программное обеспечение и драйвера устанавливается сразу, предполагается автоматическое обновление;
- 6) имеется специализированный менеджер пакетов, через который устанавливается большинство программ;
- 7) малотребовательная к ресурсам компьютера операционная система;
- 8) модульность, пользователь может изменять структуру и компоненты операционной системы, вплоть до самого ядра.

Операционная система Ubuntu состоит из 8 основных частей:

- 1) начальный загрузчик Grub, который обрабатывает процесс непосредственной загрузки ОС, задает параметры командной строки;
- 2) ядро Linux является центральной частью системы, управляет процессором, памятью и устройствами ввода/вывода;
- 3) демоны — программы, запускаемые самой системой и работающие в фоновом режиме без прямого взаимодействия с пользователем, обычно запускаются во время загрузки системы;

4) командная оболочка обеспечивает интерфейс командного процессора, что позволяет управлять компьютером с помощью ввода команд в текстовом режиме;

5) утилиты командной оболочки — в командной оболочке предоставляется некоторое количество основных встроенных команд;

6) графический сервер — часть Linux, представляющая собой графический рабочий стол, не является частью ядра Linux и реализуется с помощью пакета специального вида, известного как «Сервер X»;

7) среда рабочего стола — то, что действительно используется на рабочем столе Linux. В состав системы Ubuntu входит среда рабочего стола Unity, которая обеспечивает фон рабочего стола, панели, заголовки и границы окон;

8) программы рабочего стола — программы, работающие на рабочем столе, но не являющиеся частью среды рабочего стола.

2. Интегрированная среда разработки VS Code

2.1. Описание VS Code

Visual Studio Code (VS Code) — кроссплатформенная свободная интегрированная среда разработки, разработанная компанией Microsoft. Поддерживает разработку на широком спектре языков программирования, включая Python, C, C++, Java, JavaScript, TypeScript, C# и многие другие. Основная задача Visual Studio Code — предоставить лёгкую, но мощную среду для разработки приложений с возможностью расширения функциональности через систему плагинов. Среда интегрирована с различными системами сборки и интерпретаторами языков программирования. В редакторе кода реализовано автодополнение (IntelliSense), в том числе ключевых слов и пользовательских идентификаторов, подсветка синтаксиса, отображение сообщений об ошибках и предупреждений в реальном времени, а также возможность настройки стиля выравнивания, отступов и автоматической расстановки скобок. Встроенный отладчик используется для пошагового выполнения кода, установки точек останова, просмотра значений переменных и стека вызовов.

Visual Studio Code интегрирован с набором полезных инструментов, таких как системы управления версиями (Git, SVN), встроенный терминал для выполнения команд, эмуляторы и отладчики для различных языков программирования. Среда разработки предоставляет интерфейсы к отладчикам: для Python используется расширение Python Debugger, для C/C++ — отладчик GDB или LLDB, для JavaScript — встроенный отладчик Node.js. Через систему расширений доступна поддержка практически любого языка программирования и инструмента разработки.

Visual Studio Code предоставляет поддержку для сборки и запуска приложений на различных платформах (Windows, Linux и macOS). С помощью расширений можно организовать разработку под мобильные устройства (Android, iOS) и веб-платформы. Настройки конфигурации позволяют быстро переключаться между различными интерпретаторами, компиляторами и средами выполнения. Возможно создание файлов конфигурации для автоматизации задач сборки, тестирования и развертывания приложений.

Ключевые особенности Visual Studio Code:

1. **Встроенный терминал** — позволяет выполнять команды без переключения между окнами
2. **Расширяемость** — более 50 000 расширений в официальном маркетплейсе
3. **Интеграция с Git** — визуальное управление версиями прямо в редакторе
4. **Многоплатформенность** — единый интерфейс и функциональность на всех поддерживаемых ОС
5. **Лёгкость** — минимальное потребление ресурсов при высокой производительности
6. **Remote Development** — возможность разработки в удалённых контейнерах, WSL или SSH-сессиях

Для разработки на Python в Visual Studio Code рекомендуется устанавливать расширения: **Python**, **Pylance** (для улучшенного автодополнения и

анализа кода) и **Python Debugger** (для отладки). Эти расширения обеспечивают полный цикл разработки: от написания кода до отладки и тестирования.

2.2. Описание установки

Для установки интегрированной среды разработки Visual Studio Code в операционной системе Ubuntu можно воспользоваться Центром приложений Ubuntu (Ubuntu Software) или магазином Snap.

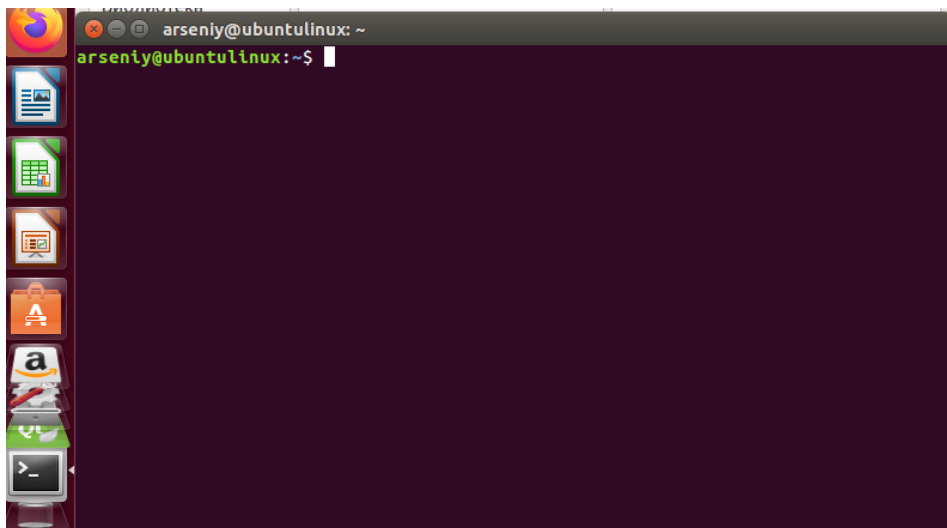


Рисунок 23 — Открытие Терминала

Visual Studio Code является кроссплатформенным редактором кода, который можно установить несколькими способами. Рассмотрим простейший способ установки через Центр приложений.

Для установки через Центр приложений необходимо:

1. Открыть меню приложений и запустить "Ubuntu Software" (Центр приложений)
2. В поисковой строке ввести "Visual Studio Code"
3. Найти приложение "Visual Studio Code" и нажать кнопку "Установить"
4. Ввести пароль пользователя для подтверждения установки

После завершения установки Visual Studio Code появится в меню приложений.

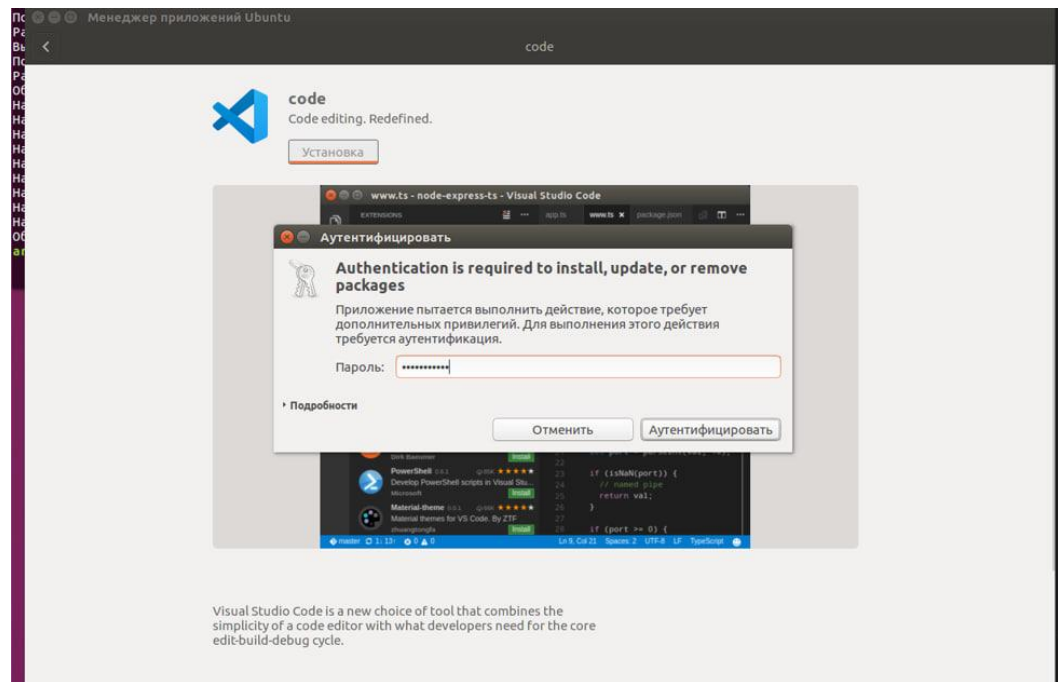


Рисунок 24 — Поиск Visual Studio Code в Центре приложений

Также возможна установка через Snap-пакет с помощью терминала командой:

text

`sudo snap install --classic code`

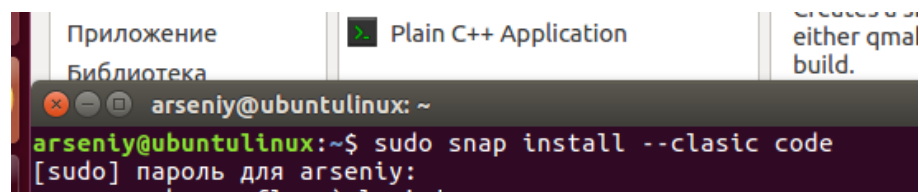


Рисунок 25 — Установка Visual Studio Code через Snap

Для разработки на Python и отладки программ рекомендуется установить следующие расширения Visual Studio Code:

1. Python (от Microsoft) — основная поддержка языка
2. Pylance — улучшенное автодополнение и анализ кода
3. Python Debugger — отладчик для Python

Установка расширений выполняется через встроенный магазин расширений (иконка "Extensions" на левой панели или комбинация клавиш Ctrl+Shift+X).

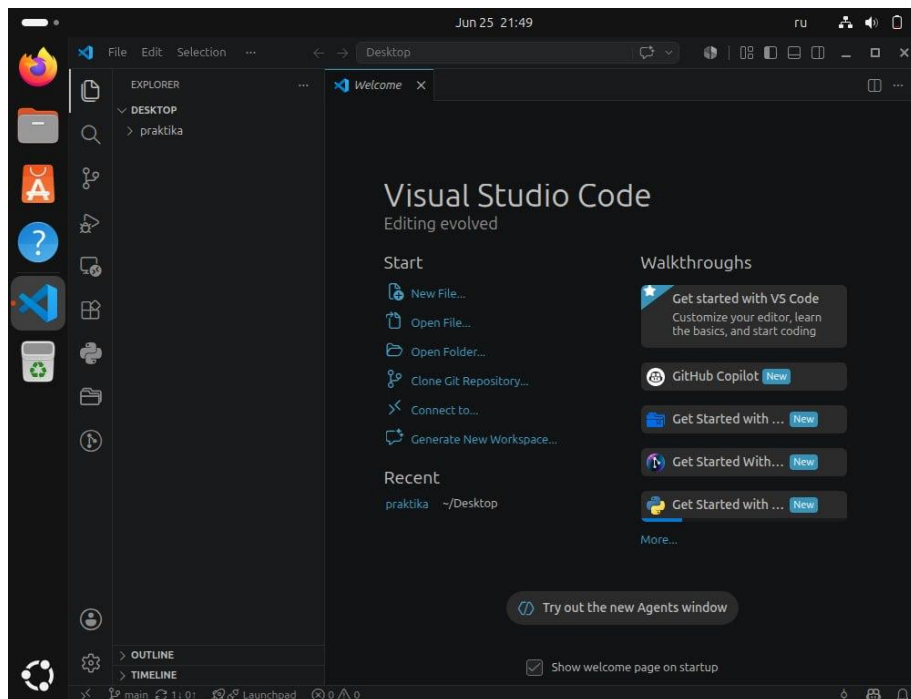


Рисунок 26 — Открытие Visual Studio Code

3. Разработка программы на языке Python в среде VS Code

3.1. Формулировка задачи

Разработать программу, которая по введенному пользователем значению аргумента x вычисляет значение функции $f(x)$, причем, при $x > 0$ $f(x) = x^{2/5} + 2x - 10$, при $x < 0$ $f(x) = 3x + 15$, при $x = 0$ $f(x) = 1$. Результат вывести на экран и записать в файл.

3.2. Работа с GitHub

Для совместной работы был создан репозиторий на GitHub.

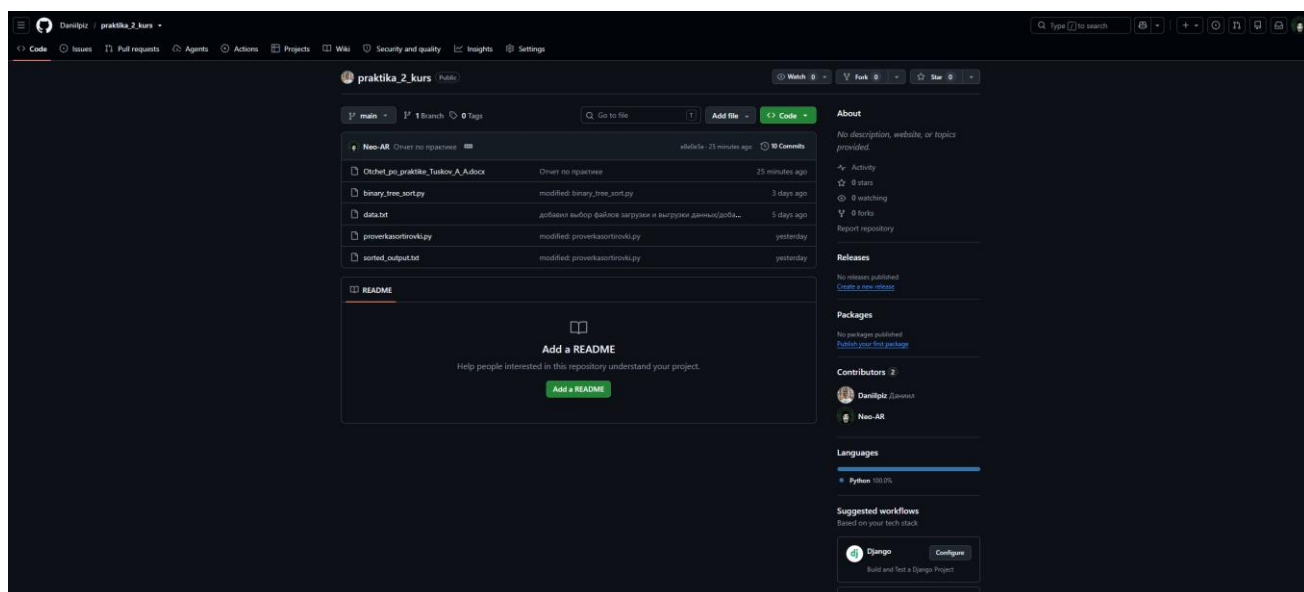


Рисунок 27 — Демонстрация репозитория

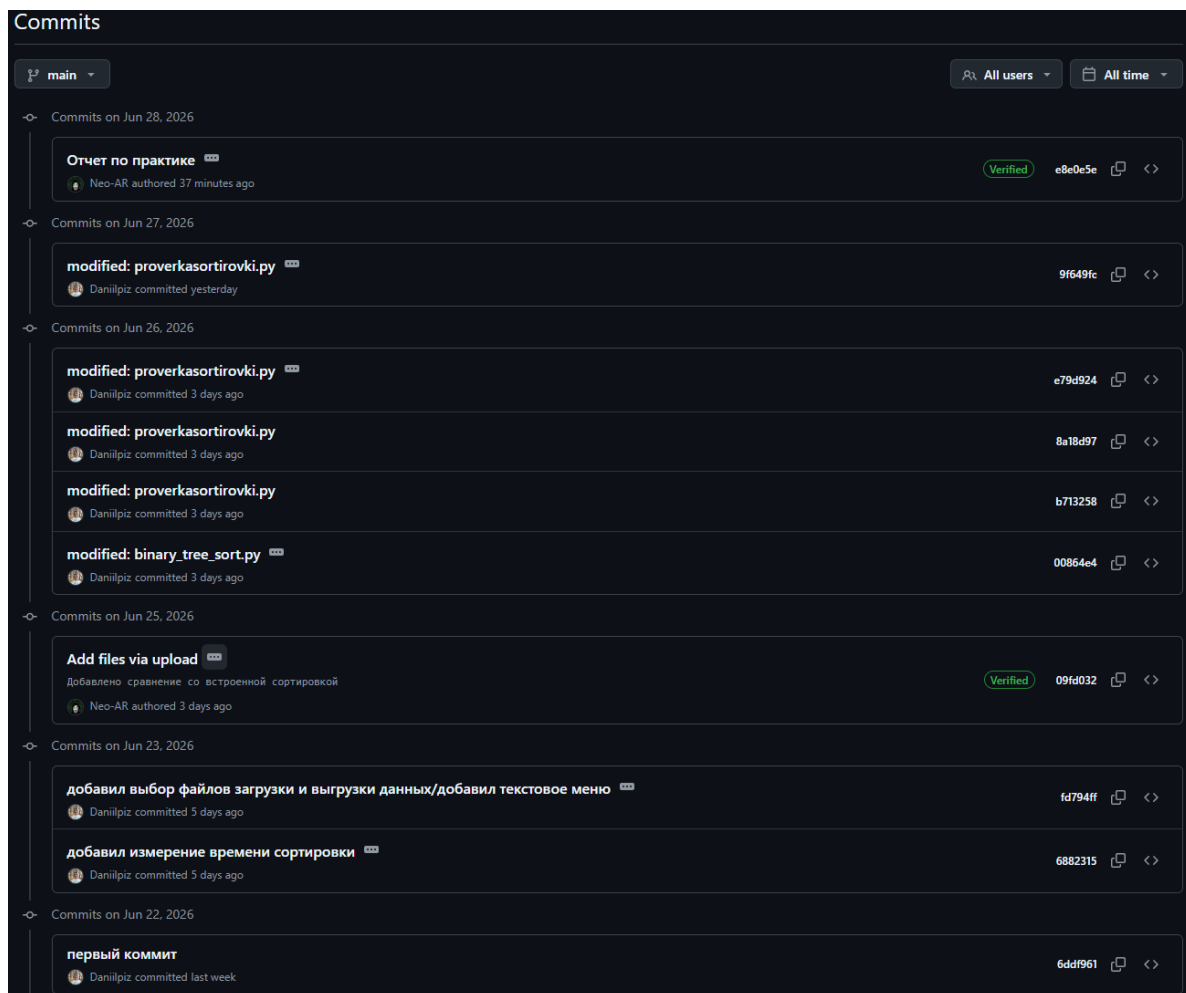


Рисунок 28 — Коммиты в репозитории

3.3. Создание проекта в VS Code

Для создания проекта в Visual Studio Code необходимо запустить среду разработки. На начальной странице открыть папку проекта через меню "File" → "Open Folder". Затем создать новый файл с расширением .py и начать написание кода программы.

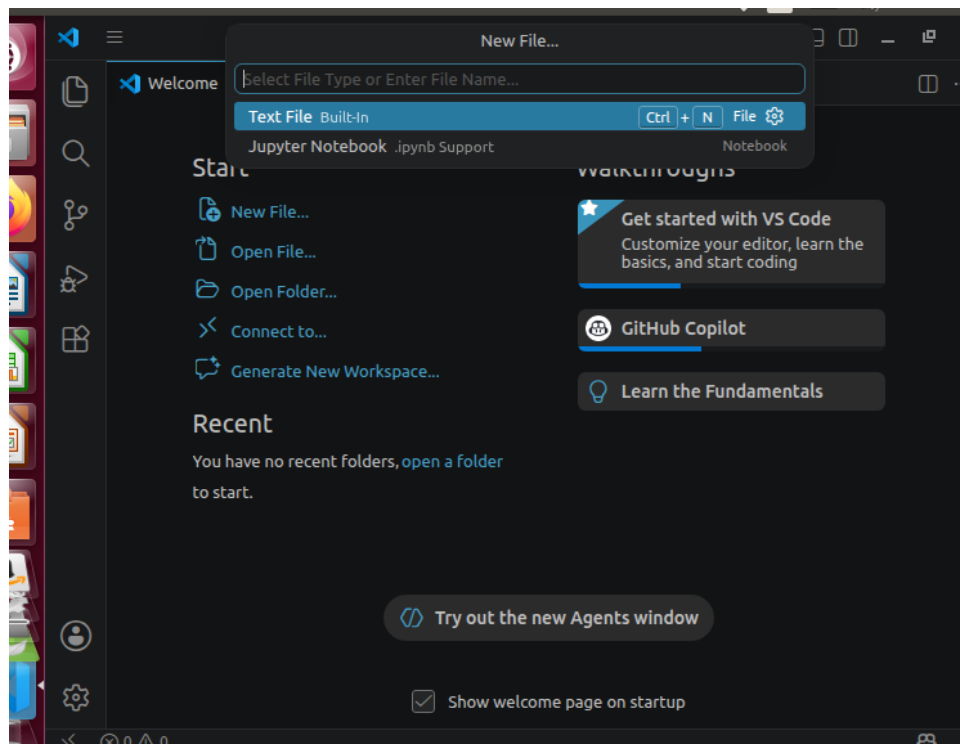


Рисунок 29 — Создание нового файла

Для создания нового файла можно использовать сочетание клавиш `Ctrl+N` или через меню "File" → "New File". После создания файла его необходимо сохранить с расширением `.py`, например `binary_tree_sort.py`.

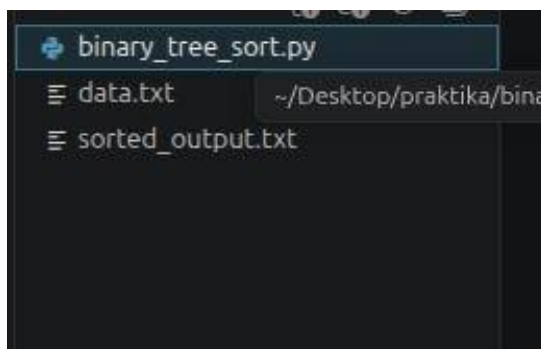


Рисунок 30 — Наш проект

Visual Studio Code автоматически определяет интерпретатор Python. При необходимости его можно выбрать через командную палитру (`Ctrl+Shift+P`) и команду "Python: Select Interpreter".

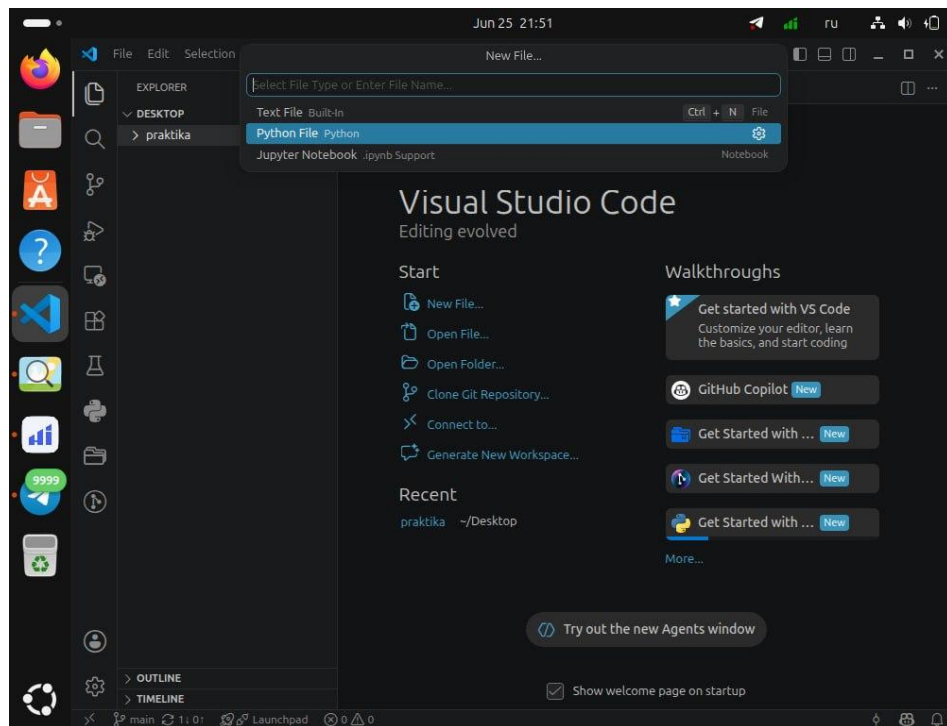


Рисунок 31 — Выбор интерпретатора Python

Для управления проектом можно использовать встроенный терминал Visual Studio Code. Открыть его можно через меню "Terminal" → "New Terminal" или сочетанием клавиш `Ctrl+``.

```
Исходный массив (13 элементов): [2, 3, 4, 4, 4, 5, 67, 67, 67, -3, -34, 0, 1]

Сохранить результат в файл?
1. Да
2. Нет (вывести в консоль)
Ваш выбор (1/2): 2

Отсортированный массив (13 элементов):
[-34, -3, 0, 1, 2, 3, 4, 4, 4, 5, 67, 67, 67]
Время сортировки: 0.000035 секунд
```

Рисунок 32 — Вывод результатов в терминал в Visual Studio Code

Проект создан, открывается редактор для написания кода программы.

```

C: > Users > Арсений > Downloads > binary_tree_sort.py
1  #!/usr/bin/env python3
2  """
3  Binary Tree Sort — интерактивным меню.
4  При загрузке из файла: Enter → data.txt в папке скрипта.
5  """
6
7  import sys
8  import os
9  import time
10
11 class TreeNode:
12     """Узел бинарного дерева поиска."""
13     def __init__(self, key):
14         self.key = key
15         self.left = None
16         self.right = None
17
18 def insert(root, key):
19     """Вставка ключа в BST. Возвращает корень дерева."""
20     if root is None:
21         return TreeNode(key)
22     if key < root.key:
23         root.left = insert(root.left, key)
24     else:
25         root.right = insert(root.right, key)
26     return root
27
28 def inorder_traversal(root, sorted_list):
29     """Симметричный обход: заполняет список отсортированными ключами."""
30     if root is not None:
31         inorder_traversal(root.left, sorted_list)
32         sorted_list.append(root.key)
33         inorder_traversal(root.right, sorted_list)
34

```

Рисунок 33 — Редактор кода Visual Studio Code

3.4. Описание программы

Программа "Binary Tree Sort" реализует алгоритм сортировки с помощью бинарного дерева поиска. Рассмотрим основные компоненты программы:

Структура `TreeNode` — узел бинарного дерева, содержит ключ (значение) и ссылки на левое и правое поддеревья. Класс инициализируется значением ключа.

Функция `insert(root, key)` — рекурсивная вставка ключа в бинарное дерево поиска. Если корень пуст, создаётся новый узел. Если ключ меньше значения корня, вставка выполняется в левое поддерево, иначе — в правое. Функция возвращает корень дерева.

Функция `inorder_traversal(root, sorted_list)` — выполняет симметричный (центрированный) обход дерева. Сначала рекурсивно обходится левое поддерево, затем текущий ключ добавляется в список результатов, после чего обходится правое поддерево. В результате формируется отсортированный список.

Функция `binary_tree_sort(arr)` — основная функция сортировки. Принимает массив чисел, последовательно вставляет каждый элемент в бинарное дерево, затем выполняет симметричный обход и возвращает отсортированный список.

Функция `read_numbers_from_file()` — загружает числа из текстового файла. Если путь не указан, используется файл `data.txt` в директории скрипта. Функция обрабатывает строки, игнорирует комментарии (строки, начинающиеся с `#`) и нечисловые значения с выводом предупреждения. Поддерживает как целые, так и вещественные числа.

Функция `manual_input()` — реализует ручной ввод чисел через терминал. Пользователь может вводить числа по одному на строке или через пробел. Ввод завершается нажатием `Enter` на пустой строке. Некорректные значения пропускаются с предупреждением.

Функция `write_output(numbers)` — управляет выводом отсортированного массива. Предлагает пользователю выбрать: сохранить результат в файл (по умолчанию `sorted_output.txt`) или вывести на экран. При выводе на экран показывается не более 50 первых элементов для больших массивов.

Функция `show_menu()` — отображает главное меню программы с тремя пунктами: загрузка из файла, ручной ввод и выход.

Функция `main()` — главный цикл программы, обрабатывающий выбор пользователя, выполняющий сортировку с замером времени и вызывающий функции вывода результата.

Программа использует модули:

1. `sys` — для работы с системными параметрами и ошибками
2. `os` — для работы с файловой системой и определения пути скрипта
3. `time` — для измерения времени выполнения сортировки

3.5. Компиляция программы в среде Visual Studio Code.

Для выполнения программы на Python в Visual Studio Code не требуется компиляция, так как Python является интерпретируемым языком. Однако среда предоставляет удобные инструменты для запуска и отладки кода.

Для запуска программы можно использовать один из следующих способов:

1. Нажать кнопку "Run" (треугольник) в правом верхнем углу редактора
2. Использовать комбинацию клавиш Ctrl + F5 (запуск без отладки)
3. Выполнить команду в терминале: `python3 binary_tree_sort.py`

```
=====
                Binary Tree Sort
=====
1. Загрузить числа из файла
2. Ввести числа вручную
3. Генерация случайных чисел
4. Выход
=====
Ваш выбор (1/2/3/4): 3
```

Рисунок 34 — Запуск программы в Visual Studio Code

```
--- Сравнение на наборах разного размера ---
Минимальное значение (Enter = 0): 5
Максимальное значение (Enter = 99999): 99999

>>> Генерация массива из 1000 чисел...
```

Рисунок 35 — Запуск программы в Visual Studio Code

```
>>> Генерация массива из 1000 чисел...

=====
                Сравнение сортировок
=====
Размер массива: 1000 элементов
Binary Tree Sort: 0.001577 сек
Встроенная sorted(): 0.000136 сек
Результаты совпадают — сортировка выполнена верно.
-----
```

Рисунок 36 — Вывод результатов сравнения сортировок

Для отладки программы в Visual Studio Code используется встроенный отладчик Python. Для запуска отладчика необходимо нажать кнопку "Run and

Debug" или использовать комбинацию клавиш F5.

Для настройки отладки создаётся файл конфигурации `.vscode/launch.json`, в котором указывается интерпретатор и параметры запуска. Можно устанавливать точки останова, щёлкая левой кнопкой мыши слева от номера строки. При достижении точки останова выполнение программы приостанавливается, и разработчик может:

1. просматривать значения переменных
2. выполнять пошаговое выполнение кода (Step Over, Step Into)
3. отслеживать стек вызовов
4. изменять значения переменных в процессе отладки

Заключение

В ходе выполнения практической работы была изучена и установлена операционная система Ubuntu на виртуальную машину VirtualBox, установлена интегрированная среда разработки VS Code в ОС Ubuntu, были изучены основные инструменты среды и разработана программа на языке Python в среде VS Code. Результаты испытаний показали, что программа работает согласно формулировке задачи.

Список использованных источников

1. Колисниченко Д.Н., Linux: Полное руководство
2. Виртуальная машина. Режим доступа:
https://ru.wikipedia.org/wiki/Виртуальная_машина
3. VirtualBox. Режим доступа: <https://ru.wikipedia.org/wiki/VirtualBox>
4. Ubuntu. Режим доступа: <https://hyperhost.ua/info/ru/ubuntu-opisanie-os-aktualnyie-versii-plyus>
5. VS Code. Режим доступа: <https://learn.microsoft.com/ru-ru/visualstudio/liveshare/use/share-project-join-session-visual-studio-code>

Приложение А

Листинг программы

Файл `proverkasortirovki.py`

```
import sys
import os
import time
import random

class TreeNode:
    """Узел бинарного дерева поиска."""
    def __init__(self, key):
        self.key = key
        self.left = None
        self.right = None

def insert(root, key):
    """Вставка ключа в BST. Возвращает корень дерева."""
    if root is None:
        return TreeNode(key)
    if key < root.key:
        root.left = insert(root.left, key)
    else:
        root.right = insert(root.right, key)
    return root

def inorder_traversal(root, sorted_list):
    """Симметричный обход: заполняет список отсортированными
    ключами."""
    if root is not None:
        inorder_traversal(root.left, sorted_list)
        sorted_list.append(root.key)
        inorder_traversal(root.right, sorted_list)

def binary_tree_sort(arr):
    """Сортировка массива через BST. Возвращает отсортированный
    список."""
    if not arr:
        return []
    root = None
    for item in arr:
        root = insert(root, item)
    result = []
    inorder_traversal(root, result)
    return result

def read_numbers_from_file():
    """Чтение чисел из файла. Enter → data.txt в папке скрипта."""
    filepath = input("Введите путь к файлу (Enter – data.txt в
    папке скрипта): ").strip()
    if not filepath:
```

```

script_dir = os.path.dirname(os.path.abspath(__file__))
filepath = os.path.join(script_dir, "data.txt")
print(f"Используется файл по умолчанию: {filepath}")

numbers = []
try:
    with open(filepath, 'r') as f:
        for line in f:
            line = line.strip()
            if not line or line.startswith('#'):
                continue
            for token in line.split():
                try:
                    num = float(token)
                    if num == int(num):
                        numbers.append(int(num))
                    else:
                        numbers.append(num)
                except ValueError:
                    print(f"Предупреждение: пропущено
нечисловое значение: '{token}'", file=sys.stderr)
            if not numbers:
                print(f"Ошибка: файл '{filepath}' не содержит чисел.")
                return None
        return numbers
except FileNotFoundError:
    print(f"Ошибка: файл '{filepath}' не найден.")
    return None
except PermissionError:
    print(f"Ошибка: нет прав для чтения файла '{filepath}'.")
    return None
except Exception as e:
    print(f"Ошибка при чтении файла: {e}.")
    return None

def manual_input():
    """Ручной ввод чисел через терминал."""
    print("\nВведите числа через пробел или по одному в строке.")
    print("Для завершения ввода нажмите Enter на пустой строке.\n")
    numbers = []
    while True:
        try:
            line = input()
        except EOFError:
            break
        if not line.strip():
            if numbers:
                break
            else:
                print("Вы ещё не ввели ни одного числа. Введите
числа или нажмите Ctrl+C для выхода.")
                continue
        for token in line.split():
            try:

```

```

        num = float(token)
        numbers.append(int(num) if num == int(num) else
num)
    except ValueError:
        print(f"Предупреждение: '{token}' – не число,
пропущено.", file=sys.stderr)
    return numbers

def compare_sorts(data, show_array=True):
    """
    Сравнивает Binary Tree Sort и встроенную sorted() по времени и
    корректности.
    Если show_array=True, выводит полный отсортированный массив.
    """
    if not data:
        return

    data_copy = data[:]
    start = time.perf_counter()
    sorted_bt = binary_tree_sort(data_copy)
    bt_time = time.perf_counter() - start

    start = time.perf_counter()
    sorted_builtin = sorted(data)
    builtin_time = time.perf_counter() - start

    correct = (sorted_bt == sorted_builtin)

    print("\n" + "=" * 50)
    print("          Сравнение сортировок")
    print("=" * 50)
    print(f"Размер массива: {len(data)} элементов")
    print(f"Binary Tree Sort: {bt_time:.6f} сек")
    print(f"Встроенная sorted(): {builtin_time:.6f} сек")
    if correct:
        print("Результаты совпадают – сортировка выполнена верно.")
    else:
        print("ОШИБКА: результаты не совпадают!")
    print("-" * 50)
    if show_array:
        print("Отсортированный массив (полностью):")
        print(sorted_bt)
        print("-" * 50)

def compare_multiple_sizes():
    """Сравнение на наборах 1K, 10K, 100K чисел."""
    print("\n--- Сравнение на наборах разного размера ---")
    min_val = input("Минимальное значение (Enter = 0): ").strip()
    max_val = input("Максимальное значение (Enter = 99999):
").strip()

    min_val = int(min_val) if min_val else 0
    max_val = int(max_val) if max_val else 99999
    if min_val > max_val:

```

```

        min_val, max_val = max_val, min_val

    sizes = [1000, 10000, 100000, 1000000] # 1K, 10K, 100K, 1M
    for size in sizes:
        print(f"\n>>> Генерация массива из {size} чисел...")
        data = [random.randint(min_val, max_val) for _ in
range(size)]
        compare_sorts(data, show_array=False) # без вывода массива
        print()

def write_output(numbers):
    """Запись отсортированных чисел в файл или вывод в консоль
(полностью)."""
    print("\nСохранить результат в файл?")
    print("1. Да")
    print("2. Нет (вывести в консоль)")
    while True:
        choice = input("Ваш выбор (1/2): ").strip()
        if choice == '1':
            while True:
                filepath = input("Введите путь для сохранения
(Enter – sorted_output.txt в папке скрипта): ").strip()
                if not filepath:
                    script_dir =
os.path.dirname(os.path.abspath(__file__))
                    filepath = os.path.join(script_dir,
"sorted_output.txt")
                    print(f"Путь не указан. Файл будет создан как
'{filepath}'")
                    try:
                        with open(filepath, 'w') as f:
                            for num in numbers:
                                f.write(f"{num}\n")
                            print(f"Результат успешно записан в
'{filepath}' ({len(numbers)} чисел).")
                            return
                    except PermissionError:
                        print(f"Ошибка: нет прав для записи в
'{filepath}'. Попробуйте снова.")
                    except Exception as e:
                        print(f"Ошибка при записи файла: {e}.
Попробуйте снова.")
                elif choice == '2':
                    print(f"\nОтсортированный массив ({len(numbers)}
элементов):")
                    print(numbers)
                    return
                else:
                    print("Неверный выбор. Введите 1 или 2.")

def show_menu():
    """Главное меню."""
    print("\n" + "=" * 50)
    print("
        Binary Tree Sort")

```

```

print("=" * 50)
print("1. Загрузить числа из файла")
print("2. Ввести числа вручную")
print("3. Генерация случайных чисел")
print("4. Выход")
print("=" * 50)

def main():
    while True:
        show_menu()
        choice = input("Ваш выбор (1/2/3/4): ").strip()

        if choice == '1':
            print("\n--- Загрузка из файла ---")
            data = read_numbers_from_file()
            if data is None:
                continue
            print(f"\nИсходный массив ({len(data)} элементов):")
            print(data)
            sorted_data = binary_tree_sort(data)
            write_output(sorted_data)
            print("\nСравнение с встроенной сортировкой\n(sorted)...")
            compare_sorts(data, show_array=True)
            input("\nНажмите Enter для возврата в меню...")

        elif choice == '2':
            print("\n--- Ручной ввод ---")
            data = manual_input()
            if not data:
                print("Ошибка: нет данных для сортировки.")
                continue
            print(f"\nИсходный массив ({len(data)} элементов):")
            print(data)
            sorted_data = binary_tree_sort(data)
            write_output(sorted_data)
            print("\nСравнение с встроенной сортировкой\n(sorted)...")
            compare_sorts(data, show_array=True)
            input("\nНажмите Enter для возврата в меню...")

        elif choice == '3':
            compare_multiple_sizes()
            input("\nНажмите Enter для возврата в меню...")

        elif choice == '4':
            print("До свидания!")
            sys.exit(0)
        else:
            print("Неверный выбор. Введите 1, 2, 3 или 4.")

if __name__ == "__main__":
    try:
        main()

```



```
except KeyboardInterrupt:  
    print("\n\nПрограмма прервана пользователем.")  
    sys.exit(0)
```

Приложение Б

Результаты испытаний

```
--- Сравнение на наборах разного размера ---  
Минимальное значение (Enter = 0): 5  
Максимальное значение (Enter = 99999): 99999
```

Рисунок Б.1 — Запускаем сравнение на разных по размеру наборах

```
>>> Генерация массива из 1000 чисел...  
  
=====
```

Сравнение сортировок	
Размер массива:	1000 элементов
Binary Tree Sort:	0.001577 сек
Встроенная sorted():	0.000136 сек
Результаты совпадают — сортировка выполнена верно.	

```
=====
```

Рисунок Б.2 – Результат тестирования с массивом из 1000 чисел

```
>>> Генерация массива из 10000 чисел...  
  
=====
```

Сравнение сортировок	
Размер массива:	10000 элементов
Binary Tree Sort:	0.025526 сек
Встроенная sorted():	0.001496 сек
Результаты совпадают — сортировка выполнена верно.	

```
=====
```

Рисунок Б.3 – Результат тестирования с массивом из 10000 элементов

```

>>> Генерация массива из 100000 чисел...

=====
Сравнение сортировок
=====
Размер массива: 100000 элементов
Binary Tree Sort: 0.439595 сек
Встроенная sorted(): 0.023774 сек
Результаты совпадают – сортировка выполнена верно.
-----

```

Рисунок Б.4 – Результат тестирования с массивом из 10000 чисел

```

>>> Генерация массива из 1000000 чисел...

=====
Сравнение сортировок
=====
Размер массива: 1000000 элементов
Binary Tree Sort: 6.567941 сек
Встроенная sorted(): 0.349735 сек
Результаты совпадают – сортировка выполнена верно.
-----

```

Рисунок Б.5 – Результат тестирования с массивом из 100000 чисел

```

--- Загрузка из файла ---
Введите путь к файлу (Enter – data.txt в папке скрипта):
Используется файл по умолчанию: C:\Users\Арсений\Downloads\data.txt

Исходный массив (12 элементов):
[1, 2, 3, 5, 0, -34, 106, 67, 67, 67, 7, 3]

```

Рисунок Б.6 – Загрузка массива из файла data.txt.

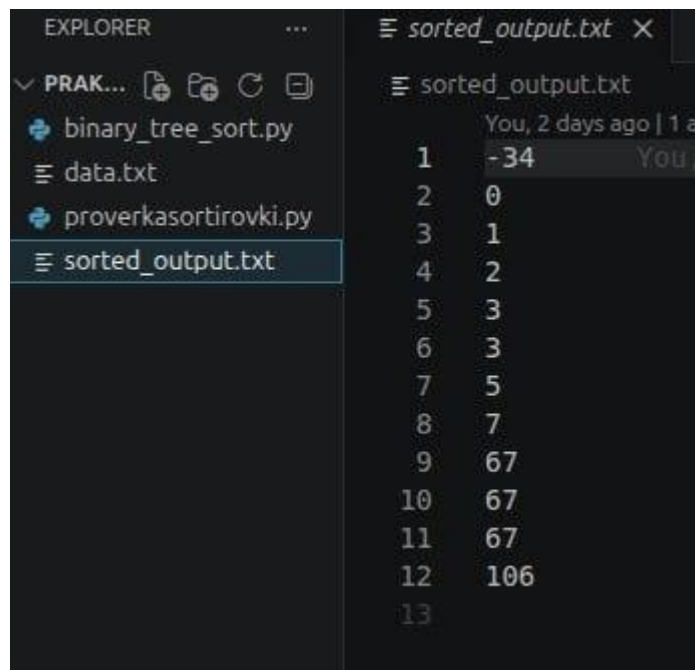


Рисунок Б.5 — Проверка записи результата в файл sorted_output.txt